

# S2D-07: Metric Creation from Logs

---

**Series:** S2D — Splunk to Dynatrace Migration | **Notebook:** 7 of 9 | **Created:** January 2026 | **Last Updated:** 06/23/2026

## Overview

---

Log-based metrics can be generated by OpenPipeline during log ingest. This provides significant benefits for frequently-executed queries used in dashboards and alerts.

This notebook explains when to request metric extraction and how to provide the necessary information.

---

## Table of Contents

---

1. [Why Create Metrics from Logs?](#)
  2. [When to Request Metric Extraction](#)
  3. [Metric Types](#)
  4. [Information Required for Metric Request](#)
  5. [Example Metric Request](#)
  6. [Best Practices](#)
  7. [Using Log-Based Metrics](#)
  8. [Request Process](#)
  9. [Metric Extraction Checklist](#)
-

## Prerequisites

---

| Requirement                  | Details                                                                |
|------------------------------|------------------------------------------------------------------------|
| <b>Dynatrace Environment</b> | SaaS with OpenPipeline                                                 |
| <b>Permissions</b>           | <code>logs.read</code> , OpenPipeline access (typically platform team) |
| <b>Knowledge</b>             | Completed SPL to DQL translation                                       |

## Learning Objectives

---

By the end of this notebook, you will be able to:

1. Understand the benefits of metric extraction
2. Identify candidates for metric creation
3. Prepare the required information for a metric request
4. Follow best practices for metric design

## Why Create Metrics from Logs?

---

Log-based metrics provide several advantages:

| Benefit                      | Description                                         |
|------------------------------|-----------------------------------------------------|
| <b>Decreased query cost</b>  | Metrics are pre-aggregated, reducing DQL processing |
| <b>Improved query speed</b>  | Metrics are optimized for time-series queries       |
| <b>Increased retention</b>   | Metrics have longer default retention than logs     |
| <b>Reduced license usage</b> | Metric queries are more efficient than log queries  |

## When to Request Metric Extraction

---

Request metric creation after SPL to DQL translation is complete.

## Good Candidates

Queries that:

- Generate timeseries results ( `makeTimeseries` or `summarize` )
- Run frequently (dashboards, alerts)
- Have stable filter criteria
- Aggregate simple counts or numeric values

## Poor Candidates

Queries that:

- Need full log content
- Use complex parsing logic
- Have frequently changing filters
- Are run infrequently (ad-hoc analysis)

## Metric Types

---

### Counter Metrics

Count the number of log records matching certain criteria.

**Use when:** You want to count occurrences (errors, requests, events)

```
// Counter metric candidate: Error log count
fetch logs, from:-24h
| filter loglevel == "ERROR"
| filter matchesPhrase(k8s.deployment.name, "checkout-service")
| makeTimeseries count = count(), by:{k8s.deployment.name}, interval:1m
```

### Value Metrics

Track a numeric value extracted from log records.

**Use when:** You want to track response times, sizes, durations, etc.

```
// Value metric candidate: Response time from logs
fetch logs, from:-24h
| filter isNotNull(response_time)
| filter matchesPhrase(k8s.deployment.name, "api-service")
| makeTimeseries avg_response = avg(response_time), by:{k8s.deployment.name},
interval:1m
```

## Information Required for Metric Request

---

### 1. Metric Type

| Type    | Description                    |
|---------|--------------------------------|
| Counter | Count of matching logs         |
| Value   | Numeric field from log records |

For **Value** metrics, specify the field containing the value (e.g., `response_time` ).

### 2. Filter Statement

The filter portion of your query.

#### Limitations:

- Must be a single continuous filter
- No custom parsing ( `parse` ) or field calculations ( `fieldsAdd` )
- Only these functions are supported:
  - `matchesPhrase`
  - `matchesValue`
  - `isNull`
  - `isNotNull`
- Basic operators and equality

### 3. Dimensions (Split By)

The `by:{{}}` parameter from your `summarize` or `makeTimeseries` command.

This defines how the metric will be segmented.

# Example Metric Request

---

## Original DQL Query

```
// Query to convert to metric
fetch logs, from:-24h
| filter loglevel == "ERROR"
| filter matchesPhrase(k8s.cluster.name, "production")
| filter matchesPhrase(k8s.namespace.name, "ecommerce")
| makeTimeseries error_count = count(), by:{k8s.deployment.name,
k8s.namespace.name}, interval:1m
```

## Metric Request Details

| Field              | Value                                                                                                                    |
|--------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Metric Type</b> | Counter                                                                                                                  |
| <b>Filter</b>      | loglevel == "ERROR" AND matchesPhrase(k8s.cluster.name, "production") AND matchesPhrase(k8s.namespace.name, "ecommerce") |
| <b>Dimensions</b>  | k8s.deployment.name , k8s.namespace.name                                                                                 |
| <b>Metric Name</b> | log.ecommerce.error_count                                                                                                |

# Best Practices

---

## Keep Metrics General

Don't create multiple similar metrics for similar alerts. Consolidate by using dimensions.

### Instead of:

- checkout\_errors
- payment\_errors
- cart\_errors

### Create:

- service\_errors with dimension k8s.deployment.name

## Avoid High-Cardinality Dimensions

Don't use unique values as dimensions:

- Transaction IDs
- Session IDs
- Request IDs
- Timestamps

These create too many metric series and impact performance.

## Consider Custom Fields

If your metric requires custom fields not available at ingest time:

1. Request a log processing rule to extract the field first
2. Then request the metric extraction using that field

## Using Log-Based Metrics

---

Once created, metrics are queried using `fetch metrics` instead of `fetch logs`:

```
// Query the extracted metric (after it's created)
// Note: This is an example - metric name will depend on your configuration
timeseries avg(log.ecommerce.error_count), from:-1h, by:{k8s.deployment.name}
```

## Request Process

---

Metric extraction is typically managed by a platform or DevOps team through OpenPipeline configuration.

### Steps

1. Complete SPL to DQL translation
2. Identify queries suitable for metric extraction
3. Document metric type, filter, and dimensions
4. Submit request to platform team
5. Wait for Terraform deployment (timing varies)

6. Update dashboards/alerts to use new metric

## Metric Extraction Checklist

---

| Item                                     | Status                   |
|------------------------------------------|--------------------------|
| DQL query validated                      | <input type="checkbox"/> |
| Metric type identified (Counter/Value)   | <input type="checkbox"/> |
| Filter uses only supported functions     | <input type="checkbox"/> |
| Dimensions defined (no high-cardinality) | <input type="checkbox"/> |
| Metric name follows naming convention    | <input type="checkbox"/> |
| Request submitted                        | <input type="checkbox"/> |

## Next Steps

---

- **S2D-08: Dashboard Migration** - Converting Splunk dashboards
- **S2D-09: Naming Standards** - Organizational conventions

## References

---

- [tutorial-log-processing-pipeline \(DT docs\)](#)
  - [Log Metrics](#)
  - [DQL Matcher Functions](#)
- 

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*